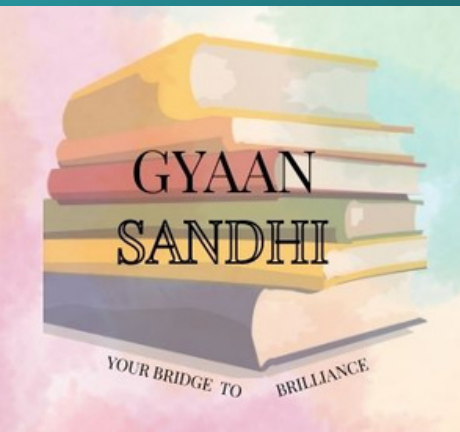




SPPU

NATURAL LANGUAGE PROCESSING

LANGUAGE MODELLING

UNIT 3

Probabilistic Language Modeling

- Probabilistic Language Modeling is a method used in Natural Language Processing (NLP) to predict how likely a sequence of words is in a language.
- It assigns a probability to a sentence or sequence of words.

Example:

- “I love ice cream” → high probability
- “Ice cream love I” → low probability
- because the first sentence follows normal English grammar and word order.

Goal

The main goal of a probabilistic language model is:

$$P(w_1, w_2, w_3, \dots, w_n)$$

which means:

Probability of a sequence of words occurring together.

Where:

- $w_1, w_2, \dots, w_n$ are words in a sentence
- n = total number of words

Chain Rule of Probability

- To calculate the probability of a whole sentence, we use the chain rule.

$$P(\mathbf{w1}, \mathbf{w2}, \dots, \mathbf{wn}) = P(\mathbf{w1}) \times P(\mathbf{w2} \mid \mathbf{w1}) \times P(\mathbf{w3} \mid \mathbf{w1}, \mathbf{w2}) \dots P(\mathbf{wn} \mid \mathbf{w1} \dots \mathbf{wn-1})$$

This means:

- Probability of first word
- × probability of second word given first word
- × probability of third word given previous words
- and so on

Problem with Full Probability Calculation

In real language:

- sentences are very long
- vocabulary is huge
- calculating probability using all previous words becomes difficult

Example:

$$P(\mathbf{wordn} \mid \mathbf{word1}, \mathbf{word2}, \dots, \mathbf{wordn-1})$$

- needs massive data and computation.

This problem is called:

- Data Sparsity Problem

Markov model

- A Markov Model is a probabilistic model used to predict the next state or next word based only on the current state.
- It is based on the Markov Assumption, which states that:
- The future state depends only on the present state and not on the past states.
- In Natural Language Processing (NLP), Markov models are used for predicting the next word in a sentence.

For example:

- If the current word is: “I”
- the model may predict: “love”
- based on probability learned from training data.

Process of Building a Simple Markov Model

- Step 1: Collect Training Sentences

Example sentences:

- I love NLP
- I love AI
- I enjoy NLP

Step 2: Create Word Pairs

Using a bigram model:

From sentence 1:

- I → love
- love → NLP

From sentence 2:

- I → love
- love → AI

From sentence 3:

- I → enjoy
- enjoy → NLP

Step 3: Count Transitions

Current	Next Word	Count
I	love	2
I	enjoy	1
love	NLP	1
love	AI	1

Step 4: Calculate Probabilities

$$P(\text{next word} \mid \text{current word}) = \frac{\text{Count}(\text{current,next})}{\text{Count}(\text{current})}$$

Example:

$$P(\text{love} \mid \text{I}) = \frac{2}{3}$$

$$P(\text{enjoy} \mid \text{I}) = \frac{1}{3}$$

Step 5: Predict Next Word

Input: “I”

The model checks probabilities and predicts: “love” because it has the highest probability.

So sentence becomes: “I love NLP” or “I love AI”

Hidden Markov Model (HMM)

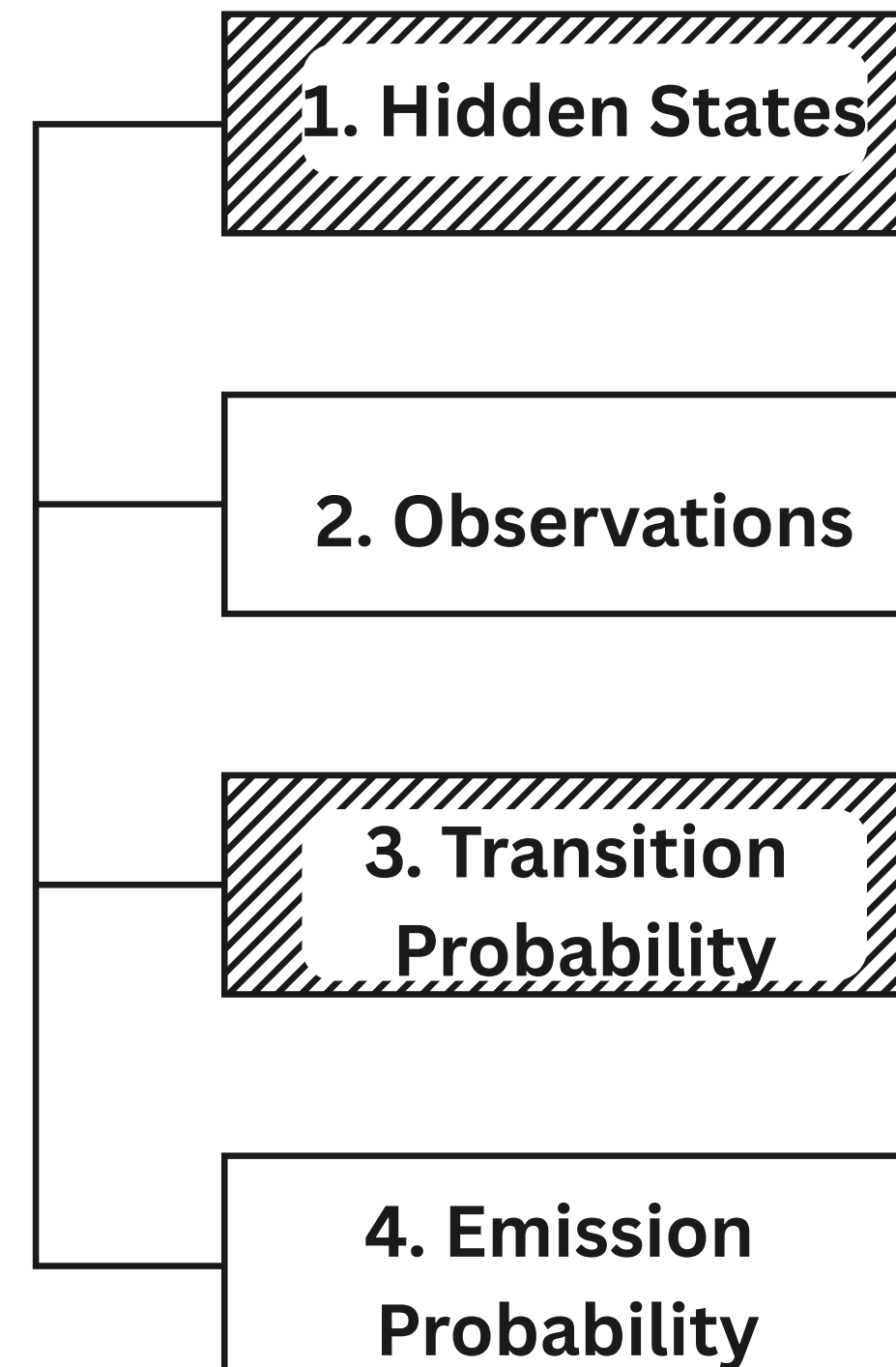
A Hidden Markov Model (HMM) is a statistical model in which the system has:

- Hidden states → cannot be directly observed
- Observable outputs → can be seen

HMM is used to predict hidden information from observed data.

It is widely used in:

- Speech recognition
- POS tagging
- NLP
- Handwriting recognition



Generative Models of Language

Generative models are models that learn how language is generated by learning the probability distribution of words and sentences.

They model:

$P(X,Y)$

where:

- X = input data
- Y = label/output

A generative model learns:

- how data is created
- relationship between words
- probability of sentences

These models can generate new text similar to training data.

Working of Generative Models

Generative models learn:

- probability of word sequences
- grammar patterns
- sentence structures

Then they predict or generate the next word based on probability.

Example:

Input:

“I love”

The model may generate:

“NLP”

because it has high probability.

Generative Model	Discriminative Model
Learns joint probability ($P(X,Y)$)	Learns conditional probability ($P(Y$
Can generate new data	Cannot generate data
Models how data is created	Models decision boundary
Used for text generation	Used for classification
Example: HMM, Naive Bayes	Example: SVM, Logistic Regression

Log-Linear Models

- used in Natural Language Processing (NLP) and Machine Learning for classification and prediction tasks.
- These models predict the probability of an output using a linear combination of features and weights.
- They are called log-linear because the logarithm of probability is represented as a linear function.

Basic Idea

A log-linear model calculates probability using:

$$P(y \mid x)$$

where:

- x = input
- y = output/class label

The model uses different features of the input and assigns weights to them.

Higher weight means greater importance of that feature.

COMPONENTS:

Features

Weights

Normalization
Factor

How Log-Linear Models Work

Step 1: Input Sentence : The model receives input text.

Example:

“I am happy”

Step 2: Extract Features

Word = happy –Positive word present

Step 3: Apply Weights : The model assigns weights to features.

Example:

“happy” → high positive weight

Step 4: Calculate Probability : Using the log-linear formula, probabilities are calculated.

Example:

Positive = 0.9

Negative = 0.1

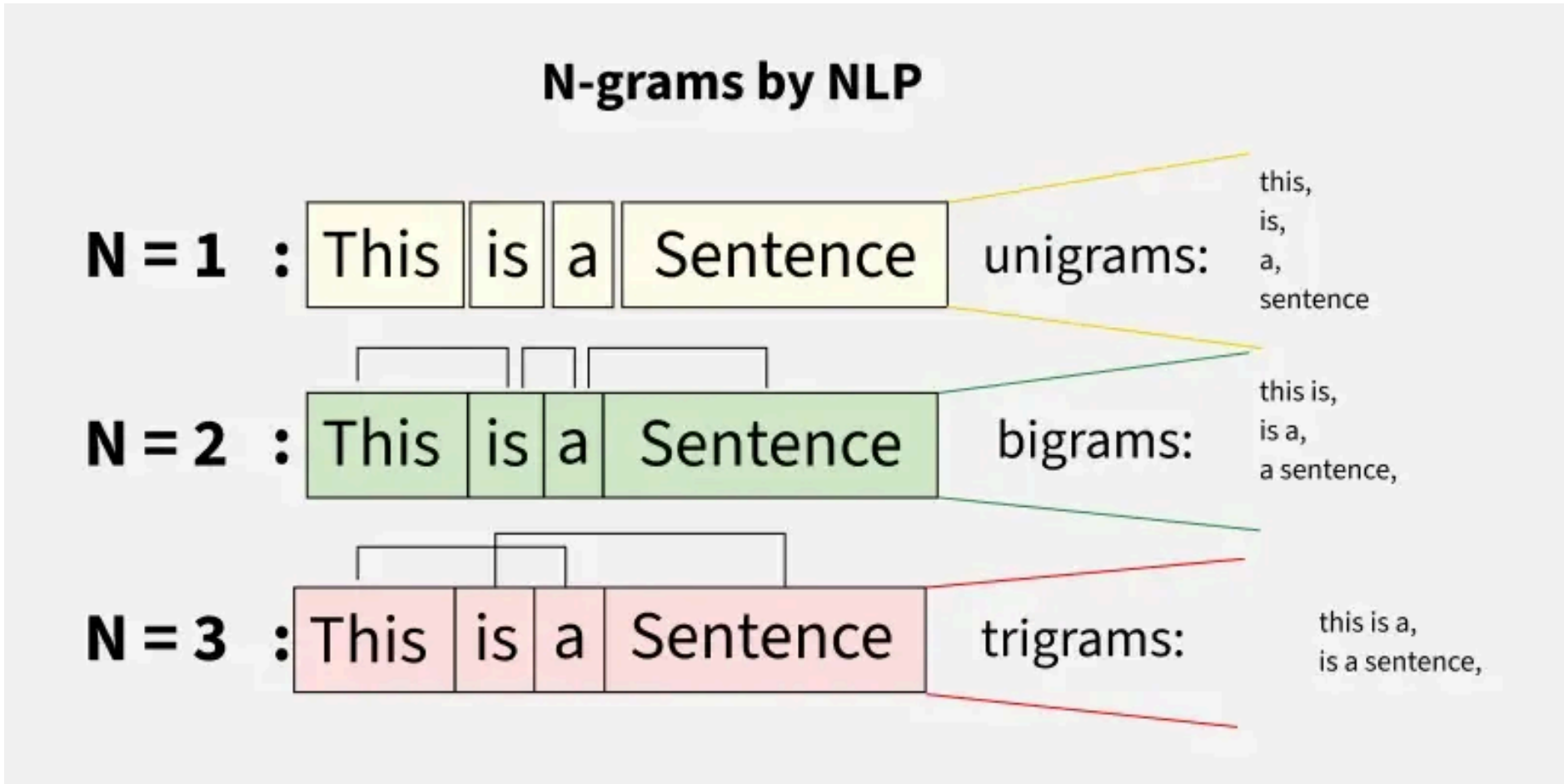
Step 5: Prediction : The class with highest probability is selected.

Result: Positive Sentiment

N-gram models

Simple N-gram Models

- An N-gram model is a probabilistic language model used in NLP to predict the next word in a sentence based on previous word(s).
- “N” represents the number of words considered together.
- N-gram models learn probabilities from text data and use them for prediction.



Type of N-gram	Meaning	Probability Formula
Unigram	Uses single word	$P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2) \dots P(w_n)$
Bigram	Uses one previous word	$P(w_n \mid w_{n-1})$
Trigram	Uses two previous words	$P(w_n \mid w_{n-2}, w_{n-1})$

b) Consider the following small corpus:

Training corpus:

<s> I am from Pune </s>

<s> I am a teacher </s>

<s> students are good and are from various cities </s>

<s> students from Pune do engineering </s>

Test data:

<s> students are from Pune </s>

Find the Bigram probability of the given test sentence.

Step 1: Bigrams in Test Sentence

<s> → students

students → are

are → from

from → Pune

Pune → *</s>*

Step 2: Bigram Probability Formula

$$P(w_n | w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n)}{\text{Count}(w_{n-1})}$$

Step 3: Calculate Each Probability

1. $P(\text{students} | <s>)$

$\text{Count}(<s> \text{ students}) = 2$

$\text{Count}(<s>) = 4$

$$P(\text{students} | <s>) = \frac{2}{4} = \frac{1}{2}$$

Step 4: Final Bigram Probability

$$P = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{2}{3} \times \frac{1}{2}$$

$$P = \frac{1}{24}$$

Final Answer

$$P(<s> \text{ students are from Pune } </s>) = \frac{1}{24}$$

OR

P=0.0416

Estimation Parameters and Smoothing

- In Natural Language Processing (NLP), language models use probabilities to predict words and sentences.
- These probabilities are calculated using parameter estimation techniques. Sometimes, some word combinations are not present in training data, causing zero probabilities. To solve this problem, smoothing techniques are used.

1. Estimation Parameters

- Parameter estimation means calculating probabilities of words or sentences from training data.
- The model estimates probabilities using frequency counts from the corpus.
- Maximum Likelihood Estimation (MLE)

Example of MLE

Training sentences:

1. I love NLP
2. I love AI
3. I enjoy NLP

Calculate:

$$P(\text{love} | I)$$

Count(I love) = 2

Count(I) = 3

So,

$$P(\text{love} | I) = \frac{2}{3}$$

This probability is estimated from training data.

Problem with Estimation

Suppose sentence:

“I study NLP”

does not exist in training data.

Then:

$$\text{Count}(I \text{ study}) = 0$$

So:

$$P(\text{study} | I) = 0$$

2. Smoothing

Smoothing techniques are used to solve zero probability problems.

Smoothing assigns small probabilities to unseen word combinations.

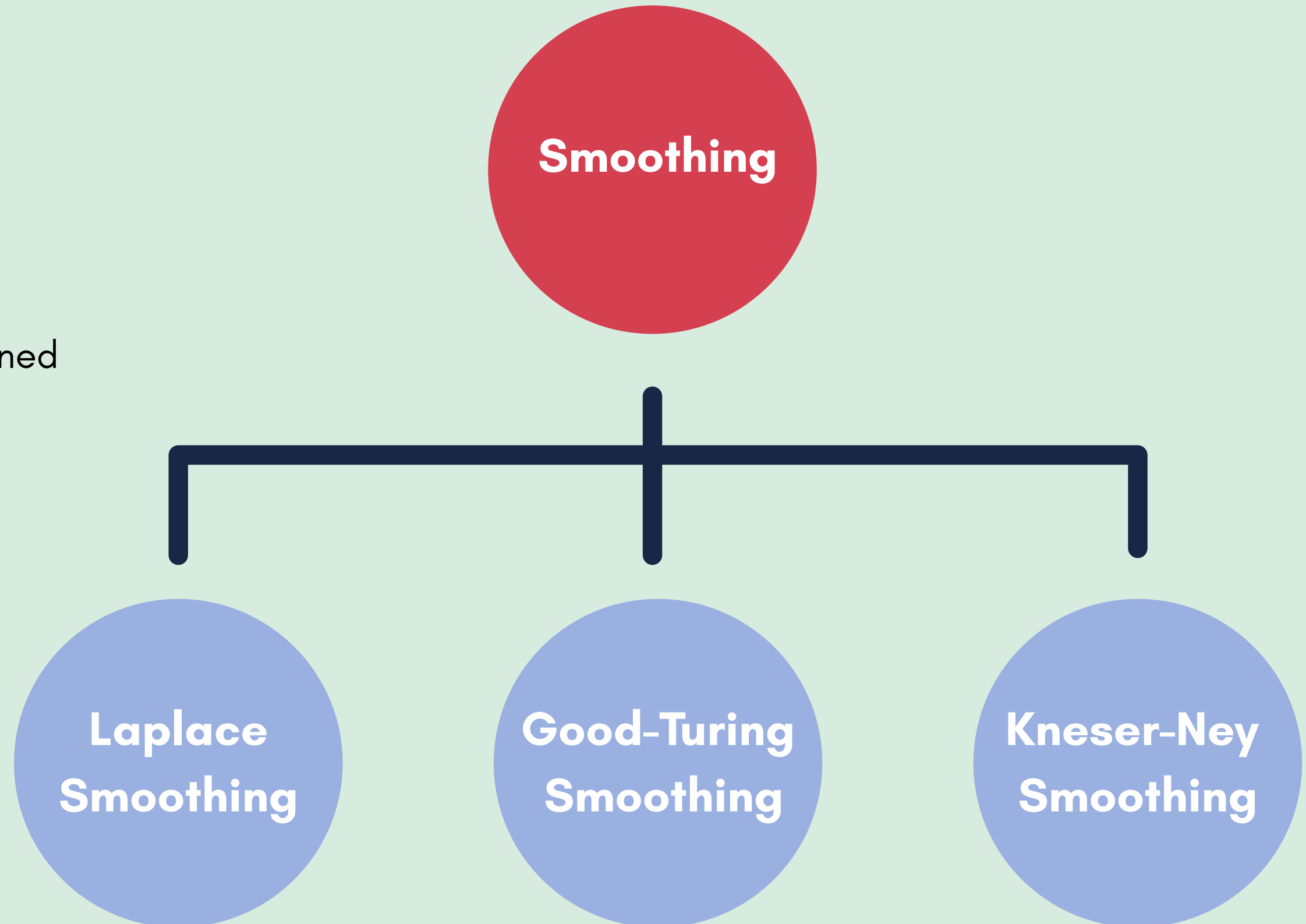
Need for Smoothing

Without smoothing:

- unseen events get probability 0
- sentence probability becomes 0

With smoothing:

- small non-zero probabilities are assigned
- model becomes more realistic



TYPES OF SMOOTHING

1. Laplace Smoothing (Add-One Smoothing)

Adds 1 to every count.

Formula:

$$P(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n) + 1}{\text{Count}(w_{n-1}) + V}$$

Where:

- V = vocabulary size

2. Good-Turing Smoothing

Adjusts probabilities based on rare events.

- Reduces probability of frequent events
- Gives some probability to unseen events

Used in speech recognition and NLP.

3. Kneser-Ney Smoothing

- An advanced smoothing technique.
- It performs well for language models by considering context and frequency.
- Widely used in modern NLP systems.

Word Embeddings/ Vector Semantics

- Word Embeddings are numerical vector representations of words used in Natural Language Processing (NLP).
- They represent words in the form of vectors so that computers can understand the meaning and relationship between words.
- Vector semantics means:
 “Words with similar meanings have similar vector representations.”

Bag-of-Words : is a text representation technique used in NLP where a document is represented by the frequency of words present in it.

It ignores:

- grammar
- word order
- sentence structure

and considers only the occurrence of words.

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Word Embeddings/ Vector Semantics

TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF is an improved text representation technique used to measure the importance of a word in a document.

It gives:

- high weight to important words
- low weight to common words

TERM FREQUENCY=
THE AMOUNT OF TIMES A
TERM APPEARS IN A
DOCUMENT



INVERSE DOCUMENT
FREQUENCY=
A MEASURE OF WHETHER A
TERM IS RARE OR COMMON
IN A COLLECTION OF
DOCUMENTS.

Bag-of-Words (BoW)	TF-IDF
Represents text using word frequency	Represents text using frequency and importance
Counts occurrence of words only	Gives weight to important words
Common words get equal importance	Common words get lower importance
Simple and easy to implement	More effective than BoW
Ignores semantic meaning	Ignores semantic meaning but improves weighting
Creates sparse vectors	Creates weighted vectors

Word Embeddings/ Vector Semantics

Word2Vec

Word2Vec is a word embedding technique used in NLP to convert words into dense vector representations.

It was developed by Google.

The main idea of Word2Vec is:

Words appearing in similar contexts have similar meanings.

It represents words as numerical vectors so that semantic relationships between words can be captured.

Word	Vector
king	[0.2, 0.8, 0.5]
queen	[0.21, 0.79, 0.48]

Doc2Vec

Doc2Vec is an extension of Word2Vec used to generate vector representations for entire documents, paragraphs, or sentences.

While Word2Vec creates vectors for words, Doc2Vec creates vectors for complete documents.

Doc2Vec represents:

- sentences
- paragraphs
- documents

as fixed-length vectors.

Word2Vec	Doc2Vec
Generates vectors for words	Generates vectors for documents
Represents individual words	Represents sentences/documents
Captures word semantics	Captures document semantics
Example: king → vector	Example: paragraph → vector

Contextualized Representations (BERT)

Contextualized representations are word representations in which the meaning of a word changes according to its context in a sentence.

Unlike traditional embeddings such as Word2Vec, contextualized models generate different vector representations for the same word in different sentences.

One of the most popular contextualized models is:

BERT

developed by **Google**.

What is BERT?

BERT stands for:

Bidirectional Encoder Representations from Transformers

It is a deep learning language model based on the Transformer architecture.

BERT reads text in both directions:

- left to right
- right to left

This helps it understand the complete context of a word.

Need for Contextualized Representations

Traditional word embeddings generate only one vector for a word.

Example:

Word: **“bank”**

Sentence 1: **“I deposited money in the bank”**

Sentence 2: **“He sat on the river bank”**

Word2Vec gives the same vector for “bank” in both sentences.

But meanings are different.

BERT generates different vectors because it understands context.

Topic Modelling: Latent Dirichlet Allocation (LDA)

Topic Modelling is a technique used in NLP to automatically discover hidden topics from a collection of documents.

One of the most popular topic modeling techniques is:

Latent Dirichlet Allocation

commonly called LDA.

LDA

What is LDA?

Latent Dirichlet Allocation (LDA) is a probabilistic model used to identify topics present in documents.

It assumes that:

- each document contains multiple topics
- each topic contains multiple words

Example:

A news article may contain topics like:

- sports
- politics
- technology

Latent

Topics are hidden and not directly visible.

Dirichlet

A probability distribution used to represent topic distribution.

Allocation

Words are allocated to different topics.

Working of LDA

Step 1: Input Documents

Suppose documents are:

1. "Cricket match and players"
2. "Election and government policies"
3. "Football team won match"

Step 2: Choose Number of Topics

Assume:

- Topic 1 → Sports
- Topic 2 → Politics

Step 3: Assign Words to Topics

Sports topic words:

- cricket
- football
- match
- players

Politics topic words:

- election
- government
- policies

Step 4: Calculate Probabilities

LDA calculates:

- probability of topics in documents
- probability of words in topics

Latent Semantic Analysis (LSA)

Latent Semantic Analysis (LSA) is a technique used in Natural Language Processing (NLP) to identify hidden relationships and meanings between words and documents.

It is used to discover semantic similarity between words based on their usage in documents.

LSA helps computers understand:

- hidden meanings
- relationships between words
- important concepts in text

Working

Step 1

Create Term-Document Matrix

A matrix is created using:

- rows $\rightarrow$ words
- columns $\rightarrow$ documents
- values $\rightarrow$ word frequencies

Step 2:

Apply Singular Value Decomposition (SVD)

LSA uses:

$$A = U \Sigma V^T$$

where:

- A = term-document matrix
- U = word matrix
- Σ = singular values
- V^T = document matrix

Step 3:

Find Hidden Semantic Relationships

Words appearing in similar contexts become semantically related.

Example:

- “car”
- “vehicle”

may become closely related in vector space.

Non-Negative Matrix Factorization (NMF)

Non-Negative Matrix Factorization (NMF) is a dimensionality reduction and topic modeling technique used in NLP and machine learning.

It decomposes a large matrix into smaller matrices containing only non-negative values.

NMF is mainly used for:

- topic modeling
- text mining
- document clustering
- feature extraction

Basic Idea of NMF

Suppose we have a matrix:

V

NMF factorizes it into two matrices:

$$\mathbf{V} \approx \mathbf{WH}$$

Where:

- V = original matrix
- W = word-topic matrix
- H = topic-document matrix

All values are non-negative.

Meaning of Matrices

1. Matrix V

Represents the term-document matrix.

- rows $\rightarrow$ words
- columns $\rightarrow$ documents

2. Matrix W

Represents relationship between:

- words and topics

3. Matrix H

Represents relationship between:

- topics and documents

Working of NMF

Step 1: Create Term-Document Matrix

Example documents:

D1: “NLP and AI”

D2: “Machine learning and NLP”

Matrix:

Word	D1	D2
NLP	1	1
AI	1	0
Machine	0	1
Learning	0	1

This forms matrix V.

Step 2: Factorize Matrix

NMF decomposes matrix:

$V \approx WH$

- word-topic matrix
- topic-document matrix

Step 3: Identify Topics

Example:

Topic 1:

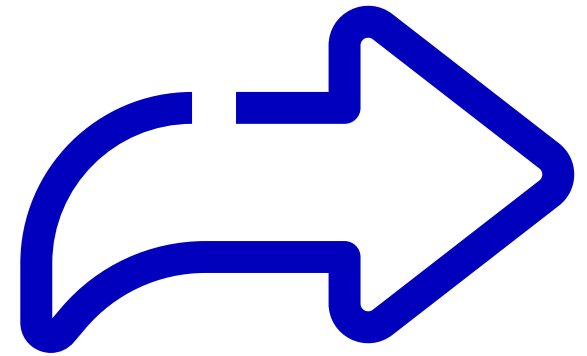
- NLP
- AI

Topic 2:

- Machine
- Learning

NMF groups related words into topics.

Feature	LSA (Latent Semantic Analysis)	LDA (Latent Dirichlet Allocation)	NMF (Non-Negative Matrix Factorization)
Full Form	Latent Semantic Analysis	Latent Dirichlet Allocation	Non-Negative Matrix Factorization
Type	Semantic analysis technique	Probabilistic topic modeling technique	Matrix factorization technique
Main Idea	Finds hidden semantic relationships	Finds hidden topics in documents	Decomposes matrix into non-negative matrices
Approach	Mathematical approach	Statistical/probabilistic approach	Linear algebra approach
Technique Used	Singular Value Decomposition (SVD)	Probability distributions	Matrix factorization
Output	Semantic concepts	Topics with word probabilities	Topics with non-negative weights
Word Values	Can contain positive and negative values	Probability values	Only non-negative values
Interpretability	Difficult to interpret	Easy to interpret topics	More interpretable than LSA
Handles Topics	Indirectly	Directly	Directly
Computational Cost	High	High	Moderate
Best Used For	Semantic similarity and search	Topic modeling	Topic extraction and clustering
Example Application	Search engines	Document topic discovery	Document clustering



Thanks for watching

